



BIRC 商智中心

Business Intelligence Center — 推動校園數位人才培育

2026 後端群體驗營

```
$ java OrderSystem --mode interactive
```

Backend 體驗營 Day2

// 訂單系統 — 從陣列到「真實世界的後端」

平行陣列
組織多筆資料

庫存判斷
避免超賣的關鍵

訂單邏輯
購物車的靈魂



昨天回顧

Day 1 學了什麼

- ✓ 變數 — 存東西的盒子
- ✓ 條件判斷 — if/else 分支
- ✓ 迴圈 — while 重複執行
- ✓ Scanner — 讀取輸入



今天要做的事

-  用平行陣列存商品資料
-  做一個超簡化的購物車
-  加入庫存判斷（避免超賣）



重點：同樣的工具（變數、if/else、while、陣列），可以做出完全不同的東西！



今天會遇到的新名詞

變數

存東西的盒子，例如 `int`
`stock = 10;`

陣列

一排盒子，例如 `int[]`
`prices = {30, 65};`

資料庫

永久保存資料的地方，例如
SQLite

if / else

判斷條件，決定走哪條路

while 迴圈

重複執行，直到條件不滿足

SQL

和資料庫溝通的語言，例如
`SELECT * FROM products`



今日目標



做出一個可互動的訂單系統

商品清單 + 選購 + 結帳



理解「庫存扣減」的邏輯

買到剩 0 件就不能再買



把資料放進真正的資料庫

建立資料表 + 新增資料 + 查詢資料



購物車的靈魂

你點外送 App 按下「加入購物車」後，後端發生了什麼？

1. 讀取商品

從資料庫拿商品清單



2. 檢查庫存

還有貨嗎？



3. 扣庫存

把它從架上拿掉



4. 計算金額

加總所有商品



這些都是後端 — 你看不到，但它一直在運作



取得範例程式碼



步驟一：陣列就是資料的容器

第一部 — 先用陣列保存商品資料

```
1  import java.util.Scanner;
2
3  // 陣列：把多筆商品資料暫時放在程式記憶體裡
4  String[] products = {"可樂", "三明治", "薯條", "餅乾", "果汁"};
5  int[] prices = {30, 65, 45, 35, 40};
6
7  // 印出商品選單
8  for (int i = 0; i < products.length; i++) {
9      System.out.println((i + 1) + ". " + products[i] + " → " + prices[i] + " 元");
10 }
11
12 // 讓使用者選購
13 Scanner scanner = new Scanner(System.in);
14 System.out.print("請輸入想買的商品編號 (1-" + products.length + ") : ");
15 int choice = scanner.nextInt();
16
17 System.out.println("你選擇了：" + products[choice - 1]);
18 System.out.println("價格：" + prices[choice - 1] + " 元");
```

陣列在做什麼？

它的限制是什麼？

1 2
3 4

陣列索引 Index 是什麼？

Step 1.5 — 陣列的「門牌號碼」

```
1 // 陣列的 index 從 0 開始！
2 String[] products = {"可樂", "三明治", "薯條", "餅乾", "果汁"};
3
4 //           ↑       ↑       ↑       ↑       ↑
5 //   index:      0       1       2       3       4
6 //   products[0] = "cola"
7 //   products[1] = "sandwich"
8 //   products[4] = "juice"      ← 最後一個是 length - 1
```

⚠ 常見錯誤

products[5] →

ArrayIndexOutOfBoundsException

index 最大只能是 length - 1

💡 為什麼從 0 開始？

index 代表「距離起點幾個位置」

第 1 個元素 = 距離起點 0 個位置



步驟二：多商品訂單

去查看範例程式碼的 `Step2.java` 邏輯

```
1  int totalAmount = 0; // 總金額
2  int orderCount = 0; // 買了幾個
3  try (Scanner scanner = new Scanner(System.in)) {
4      while (true) {
5          System.out.println("商品列表：");
6          for (int i = 0; i < products.length; i++) {
7              System.out.println((i + 1) + ". " + products[i] + " → " + prices[i] + " 元");
8          }
9          System.out.println("0. 結帳");
10         System.out.print("請選擇商品編號：");
11         int choice = scanner.nextInt();
12
13         if (choice == 0) {
14             break; // 結帳跳出迴圈
15         }
16
17         if (choice >= 1 && choice <= products.length) {
18             System.out.print("買幾個？");
19             int quantity = scanner.nextInt();
20             int subtotal = prices[choice - 1] * quantity;
21             totalAmount += subtotal;
22             orderCount += quantity;
23         }
24     }
25 }
```



迴圈的妙用



while (true) 迴圈

- • 不斷顯示商品選單
- • 讓使用者可以一直買
- • 輸入 0 才跳出

→ 這就是「購物車」的迴圈！



累加金額

- • `totalAmount += subtotal`
- • 每買一個就加一次
- • 最後就是總金額

→ 這就是「結帳」的計算！



重點：迴圈讓使用者可以一直買，`break` 讓使用者可以結帳



步驟三：庫存判斷（重點）

第一部 — 陣列裡也可以保存庫存

```
1  int[] stock = {10, 5, 8, 12, 7}; // 庫存數量
2
3  // 檢查庫存
4  if (stock[index] == 0) {
5      System.out.println("❌ " + products[index] + " 已售完，無法購買！");
6      continue; // 跳過這次，回到迴圈開頭
7  }
8  // 庫存不足
9  if (quantity > stock[index]) {
10     System.out.println("❌ 庫存不足！只剩 " + stock[index] + " 個");
11     continue;
12 }
13 // 扣庫存
14 stock[index] -= quantity;
```



第一部分會用到哪些檔案？

第一部 — 用 Java Array 完成互動式購買

專案檔案

```
day2-order-system/  
└─ BuyProductArray.java
```

資料存放的地方

資料暫時保存在 Java 記憶體裡。



如何啟動第一部分？

```
cd day2-order-system  
javac BuyProductArray.java  
java BuyProductArray
```

輸入商品編號與數量完成購買；輸入 `0` 結束。程式關閉後，陣列庫存會消失。



第二部開始：資料要保存在哪裡？

這一部要完成：把 Java 陣列換成 SQLite 資料庫



原本

商品資料在
Java 陣列裡



這一部

Java 透過 JDBC
和 SQLite 溝通

我們要做什麼？

建立 `products` 資料表，放入商品名稱、價格與庫存，接著用 Java 讀取與修改資料。

怎麼溝通？

Java 撰寫 SQL，JDBC 負責傳送 SQL；SQLite 執行後保存結果。



第二部：把資料換成資料庫

第二部 — 不換購物車邏輯，只換資料的來源



原本

資料放在 Java 陣列



現在

資料放在資料庫表格



商品名稱、價格、庫存這些「資料」不變；改變的是它們被保存、查詢的位置。

✓ 檢查點 1

✓ 我能說出

- • 陣列是什麼？
- • index 從 0 開始
- • while 迴圈做什麼？

? 我還不太確定

- • index 跟 length 的關係
- • break 跟 continue 的差別
- • 為什麼要 `stock[index] - 1`

💡 馬上複習

回到前面的「迴圈的妙用」投影片，重新看一次程式碼流程。



第二部分會用到哪些檔案？

第二部 — 用 Java JDBC 操作 SQLite

專案檔案

```
day2-order-system/  
├─ BuyProduct.java  
├─ products.db  
└─ lib/  
    └─ sqlite-jdbc-3.53.4.0.jar
```

每個檔案的用途

BuyProduct.java : Java 購買程式

products.db : 第一次執行時自動建立，保存商品與庫存

sqlite-jdbc... : 讓 Java 能連線 SQLite



如何啟動第二部分？

```
cd day2-order-system  
javac -cp "lib/sqlite-jdbc-3.53.4.0.jar" BuyProduct.java  
java -cp ".;lib/sqlite-jdbc-3.53.4.0.jar" BuyProduct
```



JDBC 五兄弟小卡

① Connection

建立資料庫連線，就像拿起電話

② PreparedStatement

準備好要問的問題 (SQL)

'?' 是先留空格，再用 `setInt()` 安全填值

③ ResultSet

資料庫回答的結果

④ `executeQuery()`

執行 SELECT，回傳 ResultSet

⑤ `executeUpdate()`

執行 INSERT/UPDATE，回傳影響筆數



用 VS Code 看資料庫

Step 2.2 — 安裝 SQLite Viewer , 直接看資料

安裝套件

1. 開啟 VS Code 的 Extensions
2. 搜尋 **SQLite Viewer**
3. 安裝作者為 **qwtel** 的套件

也可以在終端機執行：

```
code --install-extension qwtel.sqlite-viewer
```

打開資料

1. 在 VS Code 開啟 `products.db`
2. 點選資料表 `products`
3. 直接看到欄位和每一筆資料

這就是資料庫裡真正保存的資料。



Java JDBC 負責連線並執行 SQL ; SQLite Viewer 負責讓我們用表格畫面觀察資料。



Java 連線 SQLite 完成互動式購買

Step 2.5 — 列出全部商品，讓使用者選購

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.PreparedStatement;
4  import java.sql.ResultSet;
5  import java.util.Scanner;
6
7  public class BuyProduct {
8      public static void main(String[] args) throws Exception {
9          initializeDatabase();
10         try (Connection connection = DriverManager.getConnection("jdbc:sqlite:products.db");
11             Scanner scanner = new Scanner(System.in)) {
12             while (true) {
13                 printProducts(connection);
14                 System.out.print("請輸入商品 id (輸入 0 結束) :");
15                 int productId = scanner.nextInt();
16                 if (productId == 0) break;
17
18                 System.out.print("請輸入購買數量 :");
19                 int quantity = scanner.nextInt();
20                 buyProduct(connection, productId, quantity);
21             }
22         }
```

✓ 檢查點 2

✓ 我能說出

- • JDBC 是什麼？
- • Connection / PreparedStatement / ResultSet
- • products.db 裡有哪些欄位？

? 我還不太確定

- • executeQuery vs executeUpdate
- • 為什麼要 close()
- • SQL 語法怎麼寫？

💡 馬上複習

回到前面的「JDBC 五兄弟小卡」，重新看一次每個物件的用途。



第三部分會用到哪些檔案？

第三部 — 用 Java API 串接瀏覽器前端

專案檔案

```
day2-order-system/  
├ BuyProductApi.java  
├ shop.html  
├ products.db  
└ lib/  
  └ sqlite-jdbc-3.53.4.0.jar
```

每個檔案的用途

BuyProductApi.java：提供商品與購買 API

shop.html：瀏覽器看到的購物頁面

products.db：保存商品與庫存

sqlite-jdbc...：讓 API 連線 SQLite



如何啟動第三部分？

```
cd day2-order-system  
javac -cp "lib/sqlite-jdbc-3.53.4.0.jar" BuyProductApi.java  
java -cp " :lib/sqlite-jdbc-3.53.4.0.jar" BuyProductApi
```



第三部開始：讓瀏覽器加入進來

這一部要完成：用 `shop.html` 操作 Java API



shop.html

顯示商品
送出數量



Java API

接收請求
回傳 JSON



SQLite

查詢商品
更新庫存

shop.html 做什麼？

它是瀏覽器看到的購物頁面，用 JavaScript 呼叫 API，顯示商品、庫存與購買結果。

前端怎麼溝通？

GET `/api/products` 取得商品；
POST `/api/buy` 送出商品編號與數量。



第三部：Java API 串接前端

Step 3.1 — 把命令列程式變成前端可以呼叫的 API

前端

送出

`productId`、`quantity`

Java API

接收請求
執行 JDBC

SQLite

檢查庫存
扣除數量



API 版本只是把原本的 JDBC 邏輯包在 HTTP 請求裡，資料庫操作本身沒有消失。



第三部：Java API 串接前端 — 程式碼

Step 3.1 — 把命令列程式變成前端可以呼叫的 API

```
1 // BuyProductApi.java
2 // 這段是 HTTP 伺服器的固定寫法，先照抄，重點看下面的 SQL
3 server.createContext("/api/products", BuyProductApi::handleProducts);
4 server.createContext("/api/buy", BuyProductApi::handleBuy);
5
6 private static void handleBuy(HttpExchange exchange) throws IOException {
7     Map<String, String> form = parseForm(requestBody(exchange));
8     int productId = Integer.parseInt(form.get("productId"));
9     int quantity = Integer.parseInt(form.get("quantity"));
10
11     String updateSql = "UPDATE products SET stock = stock - ? "
12         + "WHERE id = ? AND stock >= ?";
13
14     try (Connection connection = DriverManager.getConnection("jdbc:sqlite:products.db");
15         PreparedStatement statement = connection.prepareStatement(updateSql)) {
16         statement.setInt(1, quantity);
17         statement.setInt(2, productId);
18         statement.setInt(3, quantity);
19         sendJson(exchange, statement.executeUpdate() == 1 ? 200 : 409,
20             "{\"message\":\"購買完成\"}");
21     }
22 }
```



這些指令在做什麼？

javac — 編譯

把人類看得懂的 .java 變成機器看得懂的 .class

```
javac -cp "lib/sqlite-jdbc-3.53.4.0.jar" BuyProductApi.java
```

`-cp` 告訴編譯器去哪裡找 JDBC 套件

java — 執行

把編譯好的 .class 跑起來

```
java -cp "lib/sqlite-jdbc-3.53.4.0.jar" BuyProductApi
```

`lib/...` 表示同時在目前目錄和 lib 資料夾找



啟動後端與前端

Step 3.2 — 編譯 API、啟動伺服器、開啟瀏覽器

先確認資料夾

```
day2-order-system/  
├─ BuyProduct.java  
├─ BuyProductApi.java  
├─ shop.html  
├─ products.db  
└─ lib/  
    └─ sqlite-jdbc-3.53.4.0.jar
```

請在 `day2-order-system/` 資料夾開啟終端機。

執行順序

1. 先編譯：產生 `BuyProduct.class`
2. 再執行：啟動 API 與前端伺服器
3. 瀏覽器開啟 `http://localhost:8080`

macOS / Linux

```
1 # 1. 編譯簡單 JDBC 版本
```



用瀏覽器完成購買

Step 3.3 — 前端串接 BuyProduct API



讀取商品

前端呼叫

GET /api/products



送出訂單

輸入數量後呼叫

POST /api/buy



更新庫存

Java 執行 SQL
前端重新載入商品



瀏覽器不直接碰 SQLite：前端呼叫 Java API，Java 再透過 JDBC 操作資料庫。

從陣列到資料庫，再到前端



前端

商品列表
輸入購買數量



Java API

接收請求
檢查庫存



SQLite

扣除庫存
保存資料

購買成功

SQLite 更新 stock，Java API 回傳結果，前端重新載入商品。

庫存不足

資料庫不執行扣除，API 回傳錯誤，前端顯示提示。



陣列是起點；真正的訂單系統，還需要前端、API 與資料庫一起合作。

檢查點 3

✓ 我能說出

- • 前端、API、資料庫的角色
- • javac 跟 java 的差別
- • 什麼是 HTTP 請求？

? 我還不太確定

- • API 端點是什麼？
- • -cp 參數為什麼要寫？
- • JSON 格式是什麼？

馬上複習

回到「這些指令在做什麼」投影片，重新看一次每個指令的用途。



恭喜！

你今天做出了一個會動的訂單系統

這就是後端的真實面貌

LucasHsu.dev — 2026 商智中心後端群體驗營